

Course Name:	Artificial Intelligence Laboratory	Semester:	VI
Date of Performance:	6/4/2026	DIV/ Batch No:	C3
Student Name:	Shreyans Tatiya	Roll No:	16010123325

Experiment No: 6

Title: Implementation of alpha-beta pruning

Aim: Implementation of Adversarial Search Algorithm - Alpha-Beta Pruning

Objectives of the Experiment:

By the end of this experiment, students will be able to:

1. Students will be able to implement and analyze the Alpha-Beta Pruning algorithm to optimize game tree search and improve decision-making efficiency

COs to be achieved:

CO2: Analyze and solve problems for goal-based agent architecture (searching and planning algorithms)

Books/ Journals/ Websites referred:

1. <https://www.geeksforgeeks.org/artificial-intelligence/alpha-beta-pruning-in-adversarial-search-algorithms/>
- 2.

Theory:

Alpha-Beta Pruning is an optimization technique used with the Minimax algorithm in game-playing scenarios such as Tic-Tac-Toe, Chess, and Checkers. The Minimax algorithm evaluates all possible moves in a game tree to determine the optimal decision for a player. However, this approach is computationally expensive as the number of nodes grows exponentially.

To overcome this limitation, Alpha-Beta Pruning reduces the number of nodes evaluated by eliminating branches that cannot influence the final decision.

Working Principle: The algorithm uses two values:

Alpha (α): The best value that the maximizing player (MAX) can guarantee so far.

Beta (β): The best value that the minimizing player (MIN) can guarantee so far.

During the traversal of the game tree: If at any node, the condition $\alpha \geq \beta$ is satisfied, further exploration of that branch is stopped. This is because that branch will not produce a better outcome than already explored options.

Adversarial Search Concept:

Adversarial search is a concept in artificial intelligence used for decision-making in environments where multiple agents (typically two players) have opposing goals. Instead of solving a problem independently, each player must consider the possible actions and counteractions of the opponent, making the situation competitive. The problem is modeled as a game tree, where nodes represent game states and edges represent moves. Algorithms like Minimax and Alpha-Beta pruning are used to evaluate these states and determine the optimal strategy by assuming that the opponent will also play optimally. This approach is commonly applied in games like chess, tic-tac-toe, and checkers, where the outcome depends on both players' decisions.

Alpha-beta pruning algorithm:

Step 1:

Start at the root node of the game tree.

Step 2:

Initialize two values:

- Alpha (α) = $-\infty$ → Best value for MAX
- Beta (β) = $+\infty$ → Best value for MIN

Step 3:

Check if the current node is a terminal node (win/loss/draw) or if the depth limit is reached:

- If yes, return the evaluated value of the node.

Step 4:

If the current node is a MAX node:

- Set bestValue = $-\infty$
- For each child node:

1. Recursively call Alpha-Beta for the child with MIN player
 2. Update:
 $\text{bestValue} = \max(\text{bestValue}, \text{returned value})$
 3. Update $\alpha = \max(\alpha, \text{bestValue})$
 4. If $\alpha \geq \beta$, stop exploring further children (prune remaining branches)
- Return bestValue

Step 5:

If the current node is a MIN node:

- Set $\text{bestValue} = +\infty$
- For each child node:
 1. Recursively call Alpha-Beta for the child with MAX player
 2. Update:
 $\text{bestValue} = \min(\text{bestValue}, \text{returned value})$
 3. Update $\beta = \min(\beta, \text{bestValue})$
 4. If $\beta \leq \alpha$, stop exploring further children (prune remaining branches)
- Return bestValue

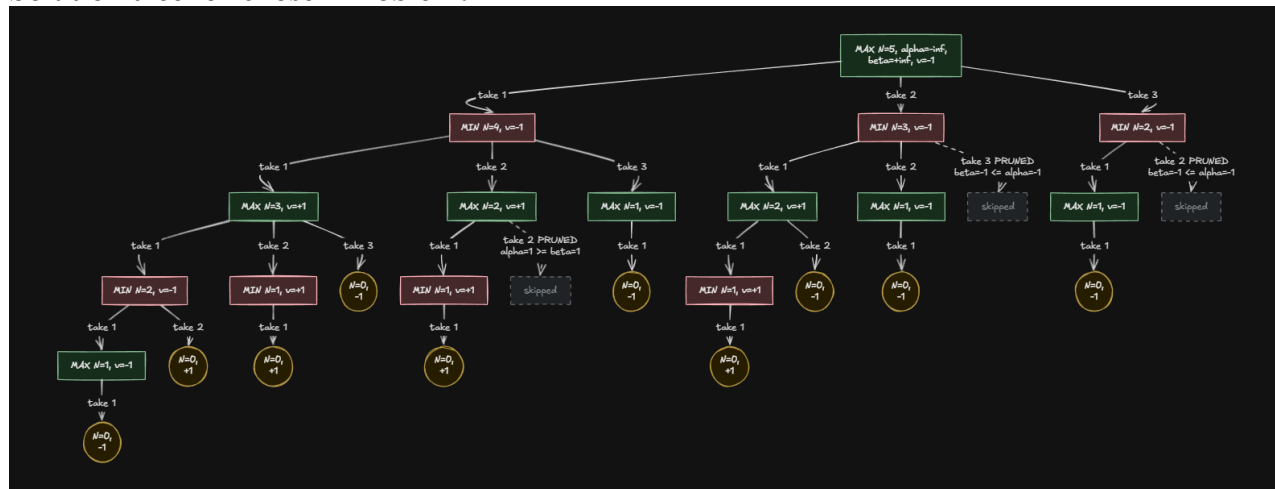
Step 6:

Continue this process recursively until the optimal value for the root node is obtained.

Chosen Problem:

Number Removal Game (Subtraction Game)

Solution tree for chosen Problem:



Code:

```
"""
```

Alpha-Beta Pruning on the Number Removal Game (Subtraction Game)

Rules:

- Start with $N = 5$.
- On each turn, the current player subtracts 1, 2, or 3 from N .
- The player who reduces N to exactly 0 WINS (i.e., takes the last).
- MAX moves first, MIN moves second. They alternate.

Leaf evaluation (terminal: $N == 0$):

- If MAX just moved (so it is MIN's turn now) -> MAX won -> +1
- If MIN just moved (so it is MAX's turn now) -> MIN won -> -1

```
"""
```

```
# Count nodes visited with and without pruning so we can show the benefit.
```

```
visited_ab = 0
```

```
visited_mm = 0
```

```
def alpha_beta(n, is_max, alpha, beta, depth=0):
```

```
    """Minimax with alpha-beta pruning on the Number Removal Game."""
```

```
    global visited_ab
```

```
    visited_ab += 1
```

```
    indent = " " * depth
```

```
    # Terminal: previous player reduced N to 0, so they won.
```

```
    # If it's MAX's turn now, MIN just moved and won -> -1.
```

```
    # If it's MIN's turn now, MAX just moved and won -> +1.
```

```
    if n == 0:
```

```
        value = +1 if is_max else -1
```

```
        print(f"{indent}[LEAF] N=0 -> value = {value:+d}")
```

```
        return value
```

```
    if is_max:
```

```
        best = float("-inf")
```

```
        print(f"{indent}MAX node at N={n} (alpha={alpha}, beta={beta})")
```

```
        for move in (1, 2, 3):
```

```
            if move > n:
```

```
                continue
```

```
            print(f"{indent} MAX tries -{move} -> N={n - move}")
```

```
            value = alpha_beta(n - move, False, alpha, beta, depth + 2)
```

```
            best = max(best, value)
```

```
            alpha = max(alpha, best)
```

```
            if alpha >= beta:
```

```
remaining = [m for m in (1, 2, 3) if m > move and m <= n]
if remaining:
    print(
        f"{indent} *** PRUNE at MAX node N={n}: "
        f"alpha={alpha} >= beta={beta}, "
        f"skipping moves {remaining} ***"
    )
    break
print(f"{indent}MAX returns {best:+d} from N={n}")
return best
else:
    best = float("-inf")
    print(f"{indent}MIN node at N={n} (alpha={alpha}, beta={beta})")
    for move in (1, 2, 3):
        if move > n:
            continue
        print(f"{indent} MIN tries -{move} -> N={n - move}")
        value = alpha_beta(n - move, True, alpha, beta, depth + 2)
        best = min(best, value)
        beta = min(beta, best)
        if beta <= alpha:
            remaining = [m for m in (1, 2, 3) if m > move and m <= n]
            if remaining:
                print(
                    f"{indent} *** PRUNE at MIN node N={n}: "
                    f"beta={beta} <= alpha={alpha}, "
                    f"skipping moves {remaining} ***"
                )
                break
            print(f"{indent}MIN returns {best:+d} from N={n}")
            return best
```

```
def plain_minimax(n, is_max):
    """Plain Minimax (no pruning) for comparison — just counts nodes visited."""
    global visited_mm
    visited_mm += 1
    if n == 0:
        return +1 if is_max else -1
    if is_max:
        best = float("-inf")
        for move in (1, 2, 3):
            if move <= n:
                best = max(best, plain_minimax(n - move, False))
        return best
```

```
else:
    best = float("+inf")
    for move in (1, 2, 3):
        if move <= n:
            best = min(best, plain_minimax(n - move, True))
    return best

def best_move_for_max(n):
    """Run alpha-beta from the root and return (best_move, best_value)."""
    best_move, best_value = None, float("-inf")
    alpha, beta = float("-inf"), float("+inf")
    for move in (1, 2, 3):
        if move > n:
            continue
        value = plain_minimax(n - move, False) # evaluated cleanly (no prints)
        if value > best_value:
            best_value = value
            best_move = move
        alpha = max(alpha, best_value)
    return best_move, best_value

if __name__ == "__main__":
    START = 5

    print("=" * 64)
    print(f" NUMBER REMOVAL GAME — starting N = {START}")
    print(" Rules: take 1, 2, or 3 on your turn. Take the last to WIN.")
    print(" MAX moves first.")
    print("=" * 64)
    print()
    print("--- Alpha-Beta Pruning trace ---")
    root_value = alpha_beta(START, is_max=True,
                            alpha=float("-inf"), beta=float("+inf"))

    print()
    print("-" * 64)
    print(f"Optimal value at root (for MAX): {root_value:+d}")
    move, _ = best_move_for_max(START)
    print(f"Optimal first move for MAX : subtract {move} "
          f"(go to N = {START - move})")
    print()

    # Node-count comparison
    visited_mm = 0
```

```
plain_minimax(START, True)
print(f"Nodes visited by plain Minimax : {visited_mm}")
print(f"Nodes visited by Alpha-Beta : {visited_ab}")
saved = visited_mm - visited_ab
pct = 100 * saved / visited_mm if visited_mm else 0
print(f"Nodes pruned : {saved} ({pct:.1f}% fewer)")
print("-" * 64)
```

Output:

```

NUMBER REMOVAL GAME - starting N = 5
Rules: take 1, 2, or 3 on your turn. Take the last to WIN.
MAX moves first.

--- Alpha-Beta Pruning trace ---
MAX node at N-5 (alpha--inf, beta--inf)
MAX tries -1 -> N-4
MIN node at N-4 (alpha--inf, beta--inf)
MIN tries -1 -> N-3
MAX node at N-3 (alpha--inf, beta--inf)
MAX tries -1 -> N-2
MIN node at N-2 (alpha--inf, beta--inf)
MIN tries -1 -> N-1
MAX node at N-1 (alpha--inf, beta--inf)
MAX tries -1 -> N-0
[LEAF] N-0 -> value - -1
MAX returns -1 from N-1
MIN tries -2 -> N-0
[LEAF] N-0 -> value - +1
MIN returns -1 from N-2
MAX tries -2 -> N-1
MIN node at N-1 (alpha--1, beta--inf)
MIN tries -1 -> N-0
[LEAF] N-0 -> value - +1
MIN returns +1 from N-1
MAX tries -3 -> N-0
[LEAF] N-0 -> value - -1
MAX returns +1 from N-3
MIN tries -2 -> N-2
MAX node at N-2 (alpha--inf, beta--1)
MAX tries -1 -> N-1
MIN node at N-1 (alpha--inf, beta--1)
MIN tries -1 -> N-0
[LEAF] N-0 -> value - +1
MIN returns +1 from N-1
*** PRUNE at MAX node N-2: alpha-1 >= beta-1, skipping moves [2] ***
MAX returns +1 from N-2
MIN tries -3 -> N-1
MAX node at N-1 (alpha--inf, beta--1)
MAX tries -1 -> N-0
[LEAF] N-0 -> value - -1
MAX returns -1 from N-1
MIN returns -1 from N-4
MAX tries -2 -> N-3
MIN node at N-3 (alpha--1, beta--inf)
MIN tries -1 -> N-2
MAX node at N-2 (alpha--1, beta--inf)
MAX tries -1 -> N-1
MIN node at N-1 (alpha--1, beta--inf)
MIN tries -1 -> N-0
[LEAF] N-0 -> value - +1
MIN returns +1 from N-1
MAX tries -2 -> N-0
[LEAF] N-0 -> value - -1
MAX returns +1 from N-2
MIN tries -2 -> N-1
MAX node at N-1 (alpha--1, beta--1)
MAX tries -1 -> N-0
[LEAF] N-0 -> value - -1
MAX returns -1 from N-1
*** PRUNE at MIN node N-3: beta--1 <= alpha--1, skipping moves [3] ***
MIN returns -1 from N-3
MAX tries -3 -> N-2
MIN node at N-2 (alpha--1, beta--inf)
MIN tries -1 -> N-1
MAX node at N-1 (alpha--1, beta--inf)
MAX tries -1 -> N-0
[LEAF] N-0 -> value - -1
MAX returns -1 from N-1
*** PRUNE at MIN node N-2: beta--1 <= alpha--1, skipping moves [2] ***
MIN returns -1 from N-2
MAX returns -1 from N-5

-----
Optimal value at root (for MAX): -1
Optimal first move for MAX : subtract 1 (go to N - 4)

Nodes visited by plain Minimax : 28
Nodes visited by Alpha-Beta : 25
Nodes pruned : 3 (10.7% fewer)

-----
--- Code Execution Successful ---

```




SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

K. J. Somaiya School of Engineering, Mumbai-77
(Somaiya Vidyavihar University)
Department of Computer Engineering



Post Lab Subjective/Objective type Questions:

1. Does Alpha–Beta pruning affect the final decision of Minimax? Explain.
Answer: No, Alpha–Beta pruning does not affect the final decision of the Minimax algorithm. It only reduces the number of nodes evaluated by eliminating branches that cannot possibly influence the final outcome. The pruning is based on already discovered bounds (alpha and beta), ensuring that any discarded branch would not produce a better result than the current best. Therefore, the optimal move selected remains exactly the same as in standard Minimax, but the computation becomes more efficient.
2. In which types of problems is Alpha–Beta pruning commonly used?
Answer: Alpha–Beta pruning is commonly used in two-player, turn-based, adversarial games with perfect information, where players have opposing goals. Typical examples include games like chess, checkers, tic-tac-toe, and connect four. These problems involve exploring a game tree of possible moves, and Alpha–Beta pruning helps optimize the Minimax algorithm by reducing unnecessary exploration, making it practical for real-time decision-making in such competitive environments.
3. How does the order of node evaluation affect the efficiency of Alpha–Beta pruning?
Answer: The efficiency of Alpha–Beta pruning heavily depends on the order in which nodes are evaluated. If the best moves are evaluated first (ideal ordering), the algorithm can prune large portions of the search tree early, significantly reducing the number of nodes explored. Conversely, if poor moves are evaluated first (worst ordering), very little pruning occurs, and the algorithm behaves similarly to standard Minimax. Hence, good move ordering greatly enhances the effectiveness of pruning.
4. What is the best-case and worst-case performance of Alpha–Beta pruning?
Answer: In the best case, when nodes are perfectly ordered, Alpha–Beta pruning reduces the time complexity from $O(b^d)$ to approximately $O(b^{(d/2)})$, effectively doubling the search depth for the same computational cost. In the worst case, when nodes are poorly ordered, no pruning occurs, and the algorithm performs like standard Minimax with time complexity $O(b^d)$. Thus, the actual performance lies between these extremes depending on how well the nodes are ordered.

Conclusion:

We successfully implemented Alpha Beta pruning for the chosen problem statement